Module 11 • System Audit & Verification

Technical Audit: IRRIGATEWISE Upgrade & Transformation

IRRIGATEWISE Hydrogel-Based Smart Irrigation System Specification

Technical Audit: IRRIGATEWISE Upgrade & Transformation

Project: IRRIGATEWISE – Hydrogel-Based Smart Irrigation System **Repository:** https://github.com/AAnnapoorani/Hydrogel-Based-Smart-Irrigation-System **Date:** September 20, 2026 **Auditor:** Antigravity AI Engineering Team **Scope:** Phase 1 Comprehensive Repository Audit & Transformation Blueprint

---

1. Executive Summary

IRRIGATEWISE was initially created as a web prototype designed for Erumaiyur Village, Tambaram, Tamil Nadu, to provide soil fertility evaluation and water conservation calculation. However, the current implementation is purely rule-based and instantaneous: - It relies on static algebraic formulas (SMD = SM_optimal - SM_current, IR = α × SMD). - It lacks any predictive foresight into soil drying dynamics. - It contains **zero hydrogel-specific logic or modeling**, despite being named "Hydrogel-Based Smart Irrigation System". - Several endpoints and pages rely on mock/dummy responses (e.g., reports return empty arrays, report export returns a dummy URL, weather widget uses fallback mocks). - Database operations are partially bypassed in demo mode.

This audit documents the existing system state and provides the architectural blueprint to transform IRRIGATEWISE into a **final-year CSE research and engineering project**: an **ML-Driven Predictive Irrigation and Agricultural Intelligence System** featuring real-time IoT ingestion, machine learning soil moisture forecasting (1h, 3h, 6h), hydrogel retention curve modeling, an automated predictive decision engine, sensor anomaly detection, pump safety automation, and closed-loop feedback verification.

---

2. Existing Architecture

2.1 Technology Stack

- **Backend:** FastAPI (Python 3.10 / 3.12 compatible) with Uvicorn.
- **ORM & Database:** SQLAlchemy 2.0.29 + PostgreSQL 16 (psycopg2-binary).
- **Authentication:** JWT tokens via PyJWT, passwords hashed via bcrypt.
- **Frontend:** Next.js 14.2.3 (App Router), React 18.3, TypeScript 5.4, Tailwind CSS 3.4.
- **UI & State:** Framer Motion 11.1, Recharts 2.12, Leaflet / React-Leaflet 4.2, Lucide React, Zustand 4.5, TanStack React Query 5.35.

2.2 Repository Directory Structure

```
Hydrogel-Based-Smart-Irrigation-System/
├── .gitignore
├── README.md
├── requirements.txt   # Unified requirements (Python + note on npm)
├── seed.sql     # SQL seed script with 58 soil and 67 irrigation records
├── backend/
│ ├── .env.example
│ ├── Dockerfile
│ ├── database.py   # DB engine, sessionmaker, Base, get_db()
│ ├── logic.py    # SoilAnalyzer, IrrigationCalculator
│ ├── main.py    # FastAPI routes, JWT auth, CORS
│ ├── models.py    # User, SoilData, IrrigationData ORM models
│ └── schemas.py    # Pydantic models for Auth, Soil, Irrigation
└── frontend/
 ├── Dockerfile
 ├── package.json
 ├── tailwind.config.js
 ├── next.config.js
 ├── tsconfig.json
 └── src/
  ├── app/
  │ ├── layout.tsx  # Root layout with fonts & providers
  │ ├── globals.css # Tailwind base styles
  │ ├── page.tsx  # Landing page with value proposition
  │ ├── dashboard/page.tsx # Dashboard with stats, chart, weather, map
  │ ├── soil/page.tsx # Soil input form & results gauge
  │ ├── irrigation/page.tsx # SMD slider calculator & scheduling
  │ └── reports/page.tsx # Historical trend & mock table/export
  ├── components/
  │ ├── Navbar.tsx  # Navigation header with responsive drawer
  │ ├── StatCard.tsx # Reusable metric card
  │ ├── WeatherWidget.tsx # Open-Meteo API consumer with fallback
  │ ├── FieldMap.tsx # Leaflet map for Erumaiyur Village field
  │ └── Providers.tsx # React Query provider wrapper
  └── lib/
   └── api.ts   # Axios client with JWT interceptors
```

---

3. Existing Modules & Code Components

Module / File	Responsibility	Current Implementation State
`backend/database.py`	PostgreSQL database connection and session lifecycle	Fully functional; defaults to local PostgreSQL connection string via `DATABASE_URL`.
`backend/models.py`	SQLAlchemy ORM tables (`users`, `soil_data`, `irrigation_data`)	Fully functional for baseline 3 tables. Missing tables for IoT readings, ML models, predictions, anomalies, feedback.
`backend/schemas.py`	Pydantic request and response schemas	Validates auth, soil input, irrigation inputs. Lacks schemas for ML, sensor readings, and decisions.
`backend/logic.py`	Domain calculations for soil scoring and irrigation deficit	Contains `SoilAnalyzer` and `IrrigationCalculator`. Strictly deterministic rule-based algorithms.
`backend/main.py`	FastAPI application endpoints	Implements basic CRUD and logic triggers. Auth is bypassed in `/soil/analyze` and `/irrigation/calculate`. Reports are stubs.
`frontend/src/lib/api.ts`	Axios instance with auth bearer token handling	Configured with `NEXT_PUBLIC_API_URL`. Attaches JWT token; redirects to `/login` on 401.
`frontend/src/app/page.tsx`	Marketing landing page	Clean aesthetic; has static statistics and email subscribe mock form.
`frontend/src/app/dashboard/page.tsx`	Main dashboard view	Displays static weekly trend, mock weather, Erumaiyur map, and static stat cards.
`frontend/src/app/soil/page.tsx`	Soil analysis interface	Form inputs for N, P, K, pH, OC, texture. Animated SVG gauge for fertility score.
`frontend/src/app/irrigation/page.tsx`	Irrigation calculator interface	Sliders for current moisture, optimal moisture, texture, field area. Schedule button.
`frontend/src/app/reports/page.tsx`	History and reporting view	Uses hardcoded static dataset in component state; export PDF button triggers mock timeout.
`frontend/src/components/WeatherWidget.tsx`	Live weather widget	Queries Open-Meteo API for Erumaiyur coordinates (12.9298° N, 80.0629° E); falls back to mock values on failure.
`frontend/src/components/FieldMap.tsx`	Interactive field map	Leaflet map with circle boundary and marker over Erumaiyur Village.

4. Existing APIs

Method	Endpoint	Auth Required	Request Payload	Response	Current Status / Limitations
`POST`	`/register`	No	`UserCreate` (name, email, password)	`TokenResponse` (token, user)	Implemented with bcrypt & JWT.
`POST`	`/login`	No	`LoginRequest` (email, password)	`TokenResponse` (token, user)	Implemented with bcrypt verification.
`POST`	`/soil/analyze`	No (in practice)	`SoilInput` (N, P, K, pH, OC, texture)	`SoilAnalysisResult`	Rule-based; skips DB save in demo mode.
`GET`	`/soil/history`	Yes	None	`List[SoilRecord]`	Queries `soil_data` by `user_id`.
`POST`	`/irrigation/calculate`	No	`IrrigationInput` (sm_current, sm_optimal, alpha, field_area)	`IrrigationResult`	Rule-based SMD calculation. No ML prediction.
`POST`	`/irrigation/schedule`	No	`ScheduleRequest` (irrigation_required, field_area, hours_from_now)	JSON message + timestamp	Mock schedule; does not persist or control hardware.
`GET`	`/irrigation/history`	Yes	None	`List[IrrigationRecord]`	Queries `irrigation_data` by `user_id`.
`GET`	`/reports`	No	None	`"soil_records": [], "irrigation_records": []`	**Stub:** Returns empty arrays!
`GET`	`/reports/export`	No	None	`"message": "...", "download_url": "..."`	**Stub:** Returns dummy URL without generating PDF/CSV.
`GET`	`/health`	No	None	`"status": "ok", "timestamp": "..."`	Working health check.

5. Existing Database Schema

5.1 Tables Present

1. **`users`** - `id` (Integer, Primary Key) - `name` (String, Not Null) - `email` (String, Unique, Indexed) - `password` (String, Hashed) - `created_at` (DateTime, default `now()`)
2. **`soil_data`** - `id` (Integer, Primary Key) - `user_id` (Integer, Foreign Key -> `users.id`) - `nitrogen` (Float) - `phosphorus` (Float) - `potassium` (Float) - `ph` (Float) - `organic_carbon` (Float) - `fertility_score` (Float) -

`classification` (String: 'High', 'Medium', 'Low') - `created_at` (DateTime, default `now()`)

3. `irrigation_data` - `id` (Integer, Primary Key) - `user_id` (Integer, Foreign Key -> `users.id`) - `sm_current` (Float) - `sm_optimal` (Float) - `smd` (Float) - `irrigation_required` (Float) - `water_saved` (Float) - `created_at` (DateTime, default `now()`)

5.2 Schema Deficiencies

- No table for time-series sensor telemetry (`sensor_readings`) containing timestamp, ambient temperature, humidity, rainfall, wind speed, hydrogel presence, soil moisture.
- No table for ML prediction tracking (`ml_predictions`) to verify forecast vs actual.
- No model metadata or version registry table (`ml_model_versions`).
- No anomaly tracking table (`anomaly_events`).
- No irrigation actuation history table (`irrigation_events`).
- No feedback loop table (`prediction_feedback`).

---

6. Existing Logic Analysis

6.1 Soil Fertility & Crop Logic (`backend/logic.py`)

- **Nutrient Scoring:**
- Score_N = min(100, (N / 120) × 100)
- Score_P = min(100, (P / 60) × 100)
- Score_K = min(100, (K / 250) × 100)
- Score_pH = 100 if 6.0 ≤ pH ≤ 7.5, 60 if 5.5 ≤ pH ≤ 8.0, else 25
- Score_OC = min(100, (OC / 0.75) × 100)
- **Weighted Fertility Score:**

> **Equation:** `Score = 0.30 · S_N + 0.20 · S_P + 0.20 · S_K + 0.15 · S_pH + 0.15 · S_OC`

Clamped between 5.0 and 98.0. - **Classification:** High (≥ 70), Medium (45 - 69), Low (< 45). - **Crop Matching:** Hardcoded array of 8 crops. Checks whether values meet minimum cutoffs; adds bonus (N - min_n)/10. Returns top 5. - **Limitation:** Rigid heuristics that do not consider weather, seasonal variations, multi-variable interactions, or probability distributions.

6.2 Irrigation Logic (`backend/logic.py`)

- **Soil Coefficients (α):**
- Clay: 1.10
- Loam: 0.85
- Sandy: 0.65
- Silt: 1.00
- **Soil Moisture Deficit (SMD):**

> **Equation:** `SMD = max(0.0, SM_optimal - SM_current)`

- **Irrigation Required (IR):**

> **Equation:** `IR = α × SMD (L/m²)`

- **Total Field Water:**

> **Equation:** `Total Water = IR × Field Area (L)`

- **Water Saved Percentage:**

> **Equation:** `Water Saved = max(0, (1 - (IR / 22.0)) × 100)`

(Using traditional irrigation baseline of 22.0\ L/m^2). - **Limitation:** Purely reactive. If moisture is 58% and optimal is 70%, it demands water immediately. If rainfall is imminent in 2 hours, or if hydrogel holds moisture and slows desorption, the formula cannot anticipate it.

---

7. What is Incomplete, Placeholder-Based, or Missing

1. **Hydrogel Mechanism Missing:** Despite the project name, there is **no code, column, or formula** representing hydrogel presence, hydrogel dosage, water absorption capacity, or desorption rates.
2. **No Machine Learning Pipeline:** Zero scikit-learn models, no training scripts, no model artifacts, no feature engineering, and no inference pipelines exist in the repository.
3. **Mock Data in Reports:** `/reports` returns empty arrays, and the frontend falls back to an in-memory array of 8 records.
4. **Mock Report Export:** `/reports/export` returns a dummy download URL string without generating a CSV or PDF.
5. **No IoT / Sensor Ingestion:** No API exists for ESP32 microcontrollers or hardware sensor nodes to publish moisture, temperature, and humidity readings.
6. **No Pump Automation Abstraction:** No controller abstraction for hardware relays or simulation mode exists.
7. **No Predictive Decision Engine:** No comparison between rule-based threshold decisions and predictive ML decisions.
8. **No Model Explainability:** No feature importance or decision attribution.
9. **Missing Docker Compose:** The README references `docker-compose.yml`, but the file is absent from the workspace root.
10. **Frontend Login Page Missing:** Although API endpoints `/login` and `/register` exist, `frontend/src/app` has no dedicated login/registration page; unauthorized users redirect to `/login` which 404s.

---

8. Machine Learning Opportunities & Target Architecture



8.1 Primary ML Model: Soil Moisture Forecasting (Time-Series Regression)

- **Targets:** `future_soil_moisture_1h`, `future_soil_moisture_3h`, `future_soil_moisture_6h`.
- **Engineered Features:**
- Moisture lag features (t-1, t-2, t-3, t-6).
- Rolling statistics (rolling mean 3h/6h, rolling std).
- Rate of change (Δ SM / Δt).
- Environmental factors: ambient temperature, relative humidity, wind speed, rainfall.
- Soil characteristics: soil texture (one-hot encoded), bulk density.

- **Hydrogel presence & concentration** (0 = control, 1 = hydrogel-treated).
- Time features: hour of day, solar position, diurnal cycle encoding (sin/cos hour).
- Irrigation history: hours since last irrigation, last irrigation volume.
- **Model Algorithms Evaluated:** 1. Linear Regression (Baseline) 2. Random Forest Regressor 3. Gradient Boosting Regressor (HistGradientBoosting / XGBoost)
- **Validation Strategy:** Chronological split (70% train, 15% validation, 15% test) to prevent temporal data leakage.
- **Metrics:** MAE, RMSE, R^2 stored in `backend/ml/artifacts/model_comparison.json`.

8.2 Hydrogel Comparative Component

- Simulates and evaluates moisture retention curves under identical environmental stress:
- **Control (without hydrogel):** Rapid moisture loss due to evapotranspiration and gravity drainage.
- **Treatment (with hydrogel):** Slower desorptive decay, prolonged moisture above permanent wilting point, extended irrigation interval.
- Dashboard visualization: Dual-curve interactive chart comparing control vs hydrogel decay.

8.3 Secondary ML Model: Crop Recommendation Classifier

- **Algorithm:** Random Forest Classifier / Gradient Boosting Classifier.
- **Inputs:** N, P, K, pH, Organic Carbon, Temperature, Humidity, Rainfall, Soil Type.
- **Outputs:** Recommended crop + prediction class probabilities.
- **Evaluation:** Precision, Recall, F1-Score, Confusion Matrix.

8.4 Sensor Anomaly Detection System

- **Algorithm:** Isolation Forest combined with domain physiological sanity rules.
- **Detections:** 1. Out-of-bounds readings (e.g. moisture <0% or >100%, extreme temperatures). 2. Sudden spikes or sensor freeze (variance = 0 over extended period). 3. Pump actuation failure (pump engaged for >10 min without subsequent moisture rise).

8.5 Predictive Irrigation Decision Engine

- **Mode 1 (Baseline):** Deterministic SMD rule (IR = α × (SM_optimal - SM_current)).
- **Mode 2 (ML Predictive):** Evaluates if predicted moisture in 3h or 6h will breach critical threshold (SM_threshold); checks weather forecast for rain in next 6h; incorporates hydrogel retention buffering; schedules proactive, optimized water delivery.
- **Output:** Comparison of both modes, volume required, pump duration, confidence, and human-readable reasoning.

8.6 Pump Controller & Hardware Safety

- `PumpController` abstraction with `SIMULATION` and `HARDWARE` modes.
- Hard safety constraints: max continuous run time (e.g. 20 min), minimum cooldown time (e.g. 60 min), emergency stop, moisture upper safety limit.

---

9. Files That Need Modification

File	Scope of Modification
`backend/models.py`	Add ORM models: `SensorReading`, `IrrigationEvent`, `MLPrediction`, `MLModelVersion`, `AnomalyEvent`, `PredictionFeedback`. Add relationships.
`backend/schemas.py`	Add Pydantic schemas for sensor ingestion, ML prediction inputs/outputs, decision engine responses, anomaly reports, and feedback.
`backend/main.py`	Mount new API routers under `/api/v1` (`/sensors`, `/ml`, `/irrigation`, `/reports`). Maintain backwards compatibility with legacy routes (`/soil/analyze`, `/irrigation/calculate`). Connect database persistence.
`backend/logic.py`	Preserve existing `SoilAnalyzer` and `IrrigationCalculator` as deterministic baselines; expose helper methods for hybrid scoring.
`backend/Dockerfile`	Update paths and ensure ML dependencies are properly installed.
`requirements.txt`	Ensure pinned versions for `scikit-learn`, `pandas`, `numpy`, `joblib`, `xgboost`, `reportlab` (for PDF export).
`frontend/src/lib/api.ts`	Add typed client methods for ML predictions, sensor streams, anomaly alerts, model versions, and irrigation decisions.
`frontend/src/app/dashboard/page.tsx`	Integrate AI Intelligence panel, live moisture forecast timeline, model performance stats, sensor health badge, and hydrogel retention curve.
`frontend/src/app/irrigation/page.tsx`	Add toggle/comparison view: "Rule-Based SMD" vs "ML Predictive Irrigation". Display predicted moisture, water saved comparison, and pump control simulator.
`frontend/src/app/soil/page.tsx`	Add toggle for "Heuristic Analysis" vs "ML Crop Recommendation" with confidence probabilities.
`frontend/src/app/reports/page.tsx`	Connect to live database backend; add ML performance report, water conservation audit, and actual CSV/PDF export.
`frontend/src/components/Navbar.tsx`	Add navigation links for ML Models and Experiments.
`README.md`	Update with complete ML architecture, API documentation, research questions, and setup guide.

10. New Files That Need to Be Created

```
backend/
├── ml/
│ ├── __init__.py
│ ├── generate_dataset.py  # Reproducible time-series simulator with hydrogel physics
│ ├── preprocessing.py   # Data cleaning, scaling, lag generation
│ ├── feature_engineering.py # Temporal, rolling, and hydrogel interaction features
│ ├── train_moisture_model.py # Multi-model training (LR, RF, GBR) & chronological eval
│ ├── train_crop_model.py  # Multi-class crop classifier training
│ ├── evaluate_models.py  # Metrics computation (MAE, RMSE, R2, F1, Confusion Matrix)
│ ├── predict.py    # Fast inference service for 1h, 3h, 6h forecasting
│ ├── anomaly_detection.py  # Isolation Forest & rule-based sensor validator
│ ├── model_registry.py   # Versioned model loader & metadata tracker
│ └── artifacts/    # Stored joblib models, scalers, and model_comparison.json
│  ├── moisture_model.joblib
│  ├── crop_model.joblib
│  ├── anomaly_model.joblib
│  └── model_comparison.json
├── services/
│ ├── __init__.py
│ ├── irrigation_decision_engine.py # Hybrid Rule-Based vs ML Predictive Decision Engine
│ ├── pump_controller.py   # Safety-constrained hardware & simulation controller
│ ├── feedback_service.py   # Closed-loop error computation & feedback logger
│ └── report_generator.py   # CSV and PDF generator for reports
├── routers/
│ ├── __init__.py
│ ├── sensors.py    # Sensor telemetry ingestion & simulator trigger
│ ├── ml.py      # ML forecasting, crop recommendation, anomaly APIs
│ ├── irrigation.py    # Predictive irrigation execution & decision history
│ └── reports.py    # Dynamic reports & file export
└── simulator/
 ├── __init__.py
 └── esp32_simulator.py  # Realistic IoT ESP32 sensor simulator

docker-compose.yml    # Docker Compose for PostgreSQL, FastAPI, and Next.js
tests/
├── __init__.py
├── test_ml_pipeline.py   # Tests for feature engineering & inference
├── test_decision_engine.py  # Tests for safety guards & hybrid decisions
├── test_anomaly_detection.py  # Tests for sensor fault identification
└── test_api_endpoints.py   # Tests for FastAPI endpoints

docs/
├── UPGRADE_AUDIT.md    # (This document)
├── ARCHITECTURE.md    # End-to-end system and data flow architecture
├── ML_PIPELINE.md    # Feature engineering, model specs, and retraining pipeline
├── DATASET.md     # Data generation methodology & hydrogel dynamics
├── MODEL_EVALUATION.md   # Model comparison and benchmark metrics
├── IRRIGATION_DECISION_ENGINE.md # Decision logic, thresholding, and safety bounds
├── IOT_INTEGRATION.md   # ESP32 protocol, payload format, and simulator guide
├── EXPERIMENTS.md    # Experimental design & empirical findings
├── API.md      # Complete OpenAPI / REST documentation
```

```
├── DEPLOYMENT.md     # Docker and cloud deployment guidelines
└── FINAL_YEAR_PROJECT_SCOPE.md # Academic project defense report & SDG alignment
```

---

11. Proposed Implementation Order

The implementation will proceed strictly in sequentially validated phases:

- **Phase 1: Project Audit & Technical Blueprint** *(Completed with this document)*
- **Phase 2: Database Schema Extension & Seed Data** (Add SQLAlchemy models, migrate tables, seed initial sensor time-series)
- **Phase 3: Dataset Generation & Hydrogel Physics Modeling** (Build reproducible generator with temporal continuity & hydrogel sorption/desorption)
- **Phase 4: ML Soil Moisture Forecasting Pipeline** (Feature engineering, train Linear Regression, Random Forest, Gradient Boosting; chronological evaluation; export `model_comparison.json`)
- **Phase 5: Model Evaluation, Artifacts & Model Registry** (Save versioned models, metadata, feature importance)
- **Phase 6: Crop Recommendation ML & Anomaly Detection Modules** (Train classifier and Isolation Forest anomaly detector)
- **Phase 7: Predictive Irrigation Decision Engine & Safety Pump Controller** (Rule-based vs ML predictive decision logic, pump simulator)
- **Phase 8: FastAPI Router Integration & Closed-Loop Feedback API** (Mount `/api/v1` routes, wire database persistence, implement feedback error tracking)
- **Phase 9: ESP32 IoT Ingestion & Hardware Simulator** (Endpoint `/api/v1/sensors/readings` and autonomous ESP32 background simulator)
- **Phase 10: Frontend Dashboard Upgrade & Visual Intelligence** (Embed AI forecast graph, hydrogel retention curves, model performance metrics, sensor health indicators)
- **Phase 11: Frontend Irrigation, Soil & Reports Integration** (Compare rule vs ML decisions, interactive simulator, live reports with CSV/PDF download)
- **Phase 12: Experiments & Documentation Suite** (Run 6 benchmark experiments, generate `docs/*.md`, update `README.md` and `docker-compose.yml`)
- **Phase 13: End-to-End Verification & Quality Assurance** (Comprehensive test suite, error boundary verification, Docker build validation)

Audit approved and ready for Phase 2 execution.